Rozdział 1

Koncepcja i Technologia Komputerów

Rewolucja Komputerowa

- Postęp w technologii komputerowej
  - Wspierana przez prawo Moore'a
- Daje w efekcie nowe aplikacje
  - Komputery w samochodach
  - Telefony komórkowe
  - Human genome project
  - World Wide Web
  - Search Engines
- Komputery są wszędzie

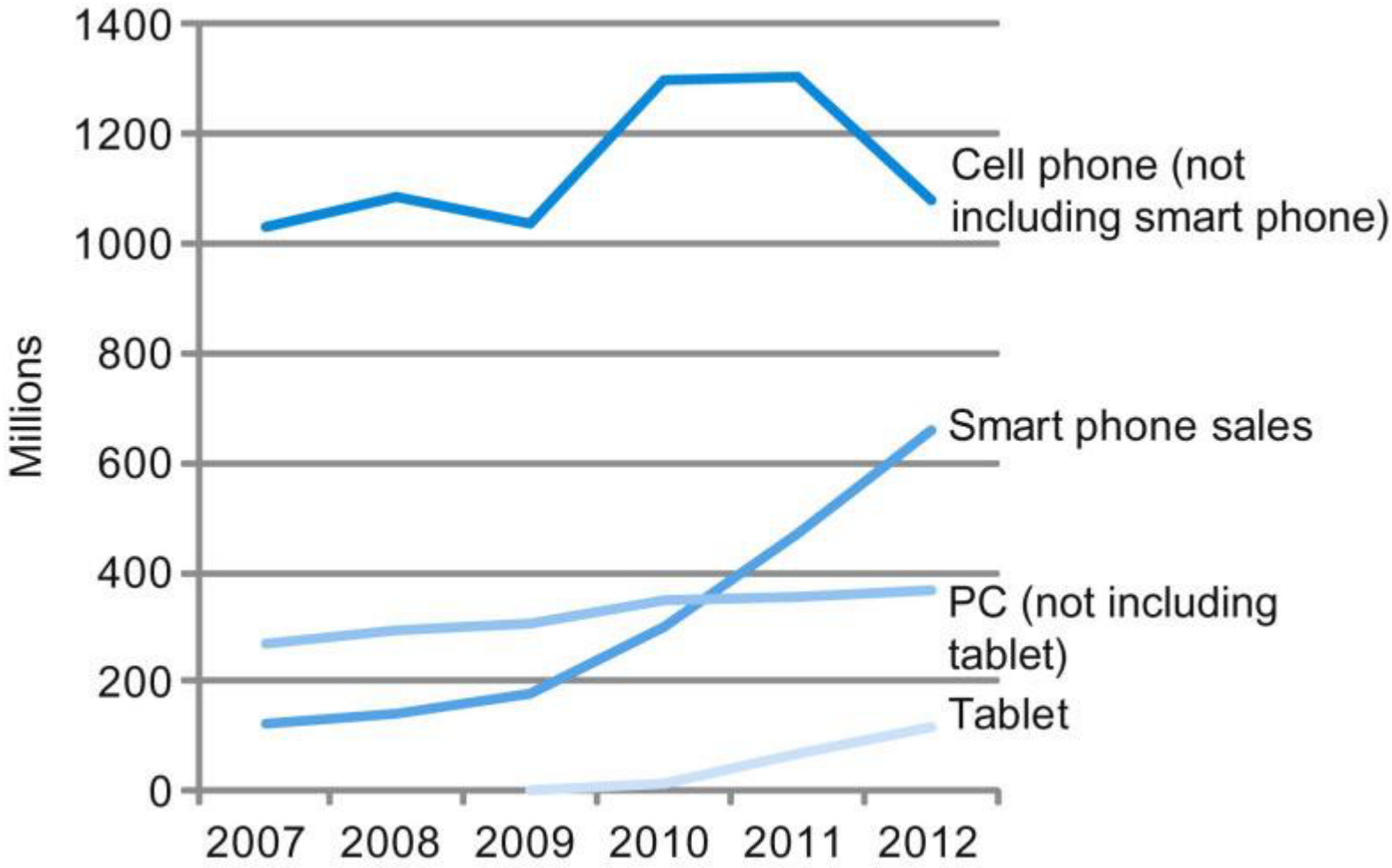
Klasy Komputerów

- Komputery personalne (Personal computers, PCs)
  - Ogólnego przeznaczenia, różnorodność oprogramowania
  - Wybór między kosztem a wydajnością
- Serwery
  - Powiązane z sieciami
  - Duża pojemność, wydajność, niezawodność
  - Od małych serwerów do „stajni”

Klasy komputerów

- Superkomputery
  - Najbardziej wymagające zadania obliczeniowe naukowe i inżynieryjne
  - Najbardziej wydajne, ale stanowią tylko drobny ułamek rynku komputerów
- Komputery zagnieżdżone (Embedded computers)
  - Ukryte jako elementy systemów
  - Ostre ograniczenia na moc/wydajność/koszt

Era PostPC


1400
1200
1000
800
600
400
200
0
Millions
2007
2008
2009
2010
2011
2012
Cell phone (not including smart phone)
Smart phone sales
PC (not including tablet)
Tablet

Era PostPC

- Personal Mobile Device (PMD)
  - Zasilane z baterii
  - Łączą się z Internetem
  - Cena – setki dolarów
  - Smartfony, tablety, okulary elektroniczne
- Obliczenia w Chmurze (Cloud computing)
  - Warehouse Scale Computers (WSC)
  - Software as a Service (SaaS)
  - Część oprogramowania wykonywane jest na PMD a część w Chmurze
  - Amazon i Google

Czego będziemy się uczyć

- W jaki sposób programy są tłumaczone na język maszynowy
  - I jak wykonuje je sprzęt (hardware)
- Interfejs sprzęt/oprogramowanie (hardware/software)
- Co decyduje o wydajności programu
  - I jak można ją poprawić
- Jak twórcy sprzętu poprawiają wydajność
- Co to jest przetwarzanie równoległe

Rozumieć pojęcie wydajności

- Algorytm
  - Określa liczbę wykonywanych operacji
- Języki programowania, kompilatory, architektura
  - Określają liczbę instrukcji maszynowych wykonywanych na operację
- Procesor i system pamięci
  - Określają jak szybko wykonywane są instrukcje
- System wejścia/wyjścia, w tym operacyjny (I/O system, including operating system – OS)
  - Określają jak szybko wykonywane są operacje we/wy

Osiem wielkich idei

- Projektuj dla ***Prawa Moore'a***
- ***Abstrahuj*** w celu uproszczenia projektu
- Czyń ***typowy przypadek szybkim***
- Wydajność przez ***równoległość***
- Wydajność przez ***potokowość***
- Wydajność przez ***predykcję***
- ***Hierarchia*** pamięci
- ***Niezawodność*** przez nadmiarowość









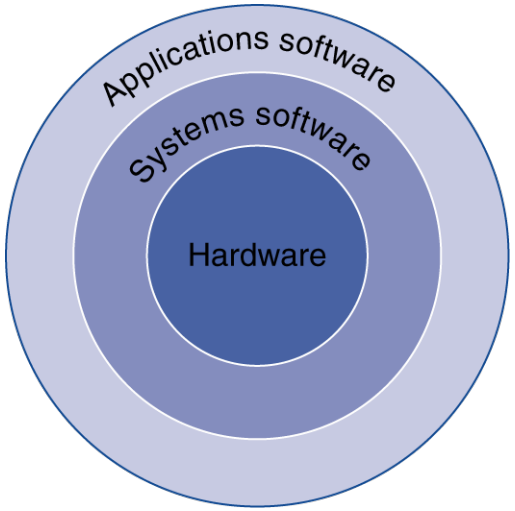








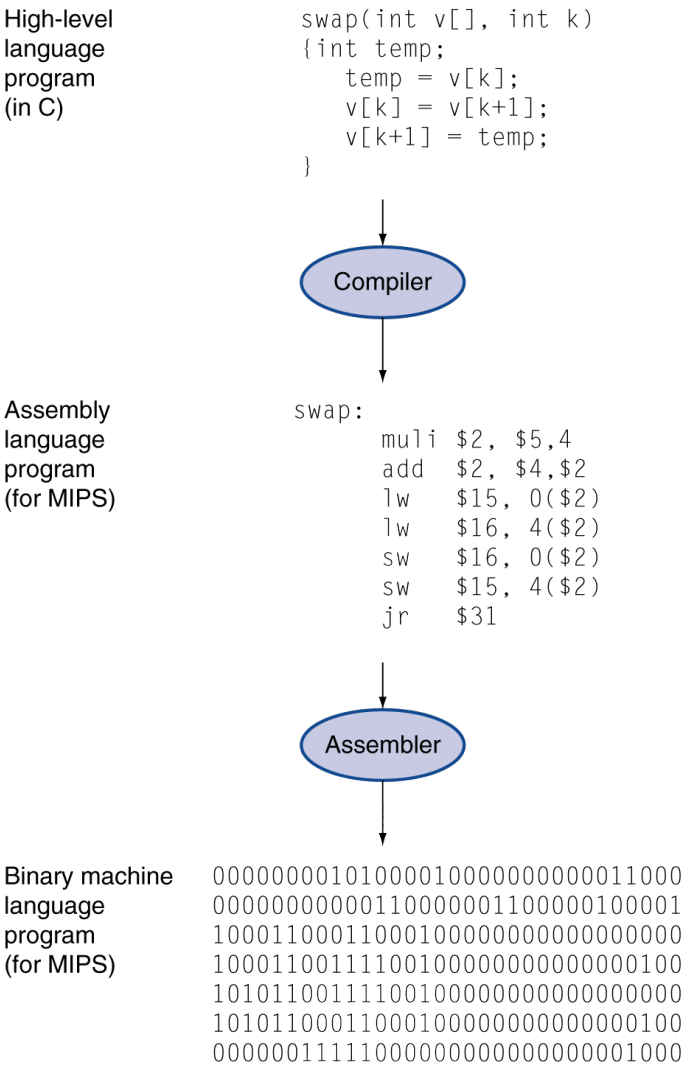
Poniżej Twojego programu



- Aplikacja (Application software)
  - Napisana w języku wysokiego poziomu (HLL, high level language)
- Oprogramowanie systemowe
  - Kompilator: tłumaczy kod HLL na kod maszynowy
  - System operacyjny: kod usługowy
    - Realizuje wejście/wyjście
    - Zarządza pamięcią
    - Szereguje zadania & dzieli zasoby
- Sprzęt (Hardware)
  - Procesor, pamięć, kontrolery we/wy

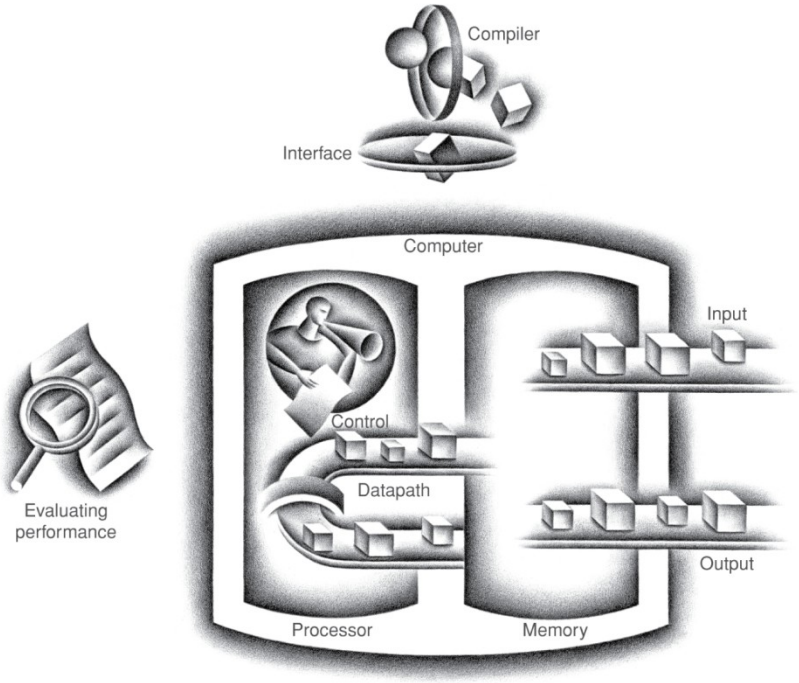
Poziomy kodu programu

- Język wysokiego poziomu
  - Poziom abstrakcji bliższy poziomu problemu
  - Stosowany dla wydajności i przenośności
- Język asemblera
  - Tekstowa reprezentacja instrukcji
- Reprezentacja sprzętowa
  - Cyfry binarne (bity)
  - Zakodowane instrukcje i dane



Części komputera

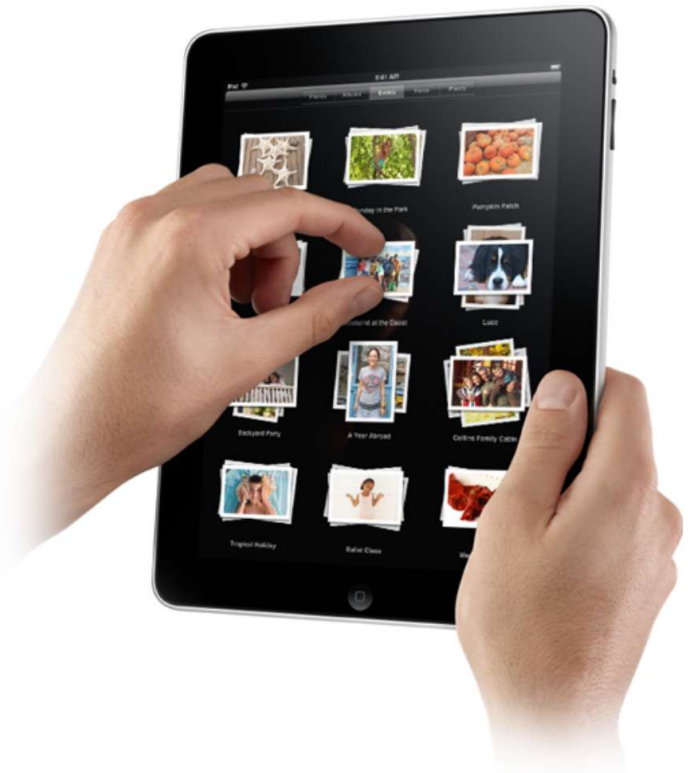
The BIG Picture



- Te same części dla wszystkich komputerów
  - Stacjonarne (desktop), serwery, zagnieżdżone
- Wejścia/wyjścia to także
  - Interfejsy użytkownika
    - Ekran, klawiatura, mysz
  - Urządzenia pamięciowe
    - Dysk twardy, CD/DVD, flash
  - Adaptery sieciowe
    - Dla komunikacji z innymi komputerami

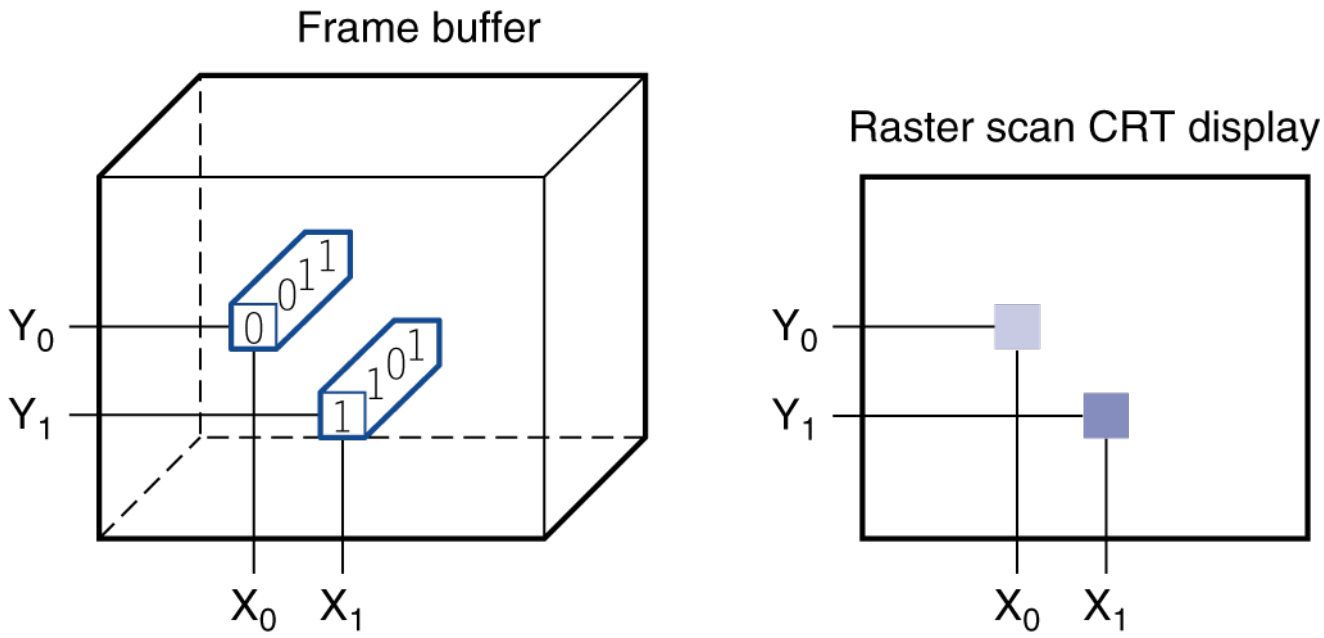
Ekran dotykowy

- Urządzenie „PostPC”
- Zastępuje klawiaturę i mysz
- Rezystancyjny lub pojemnościowy
  - Większość tabletów i smartfonów ma pojemnościowy
  - Pojemnościowe rozpoznają wiele dotknięć jednocześnie

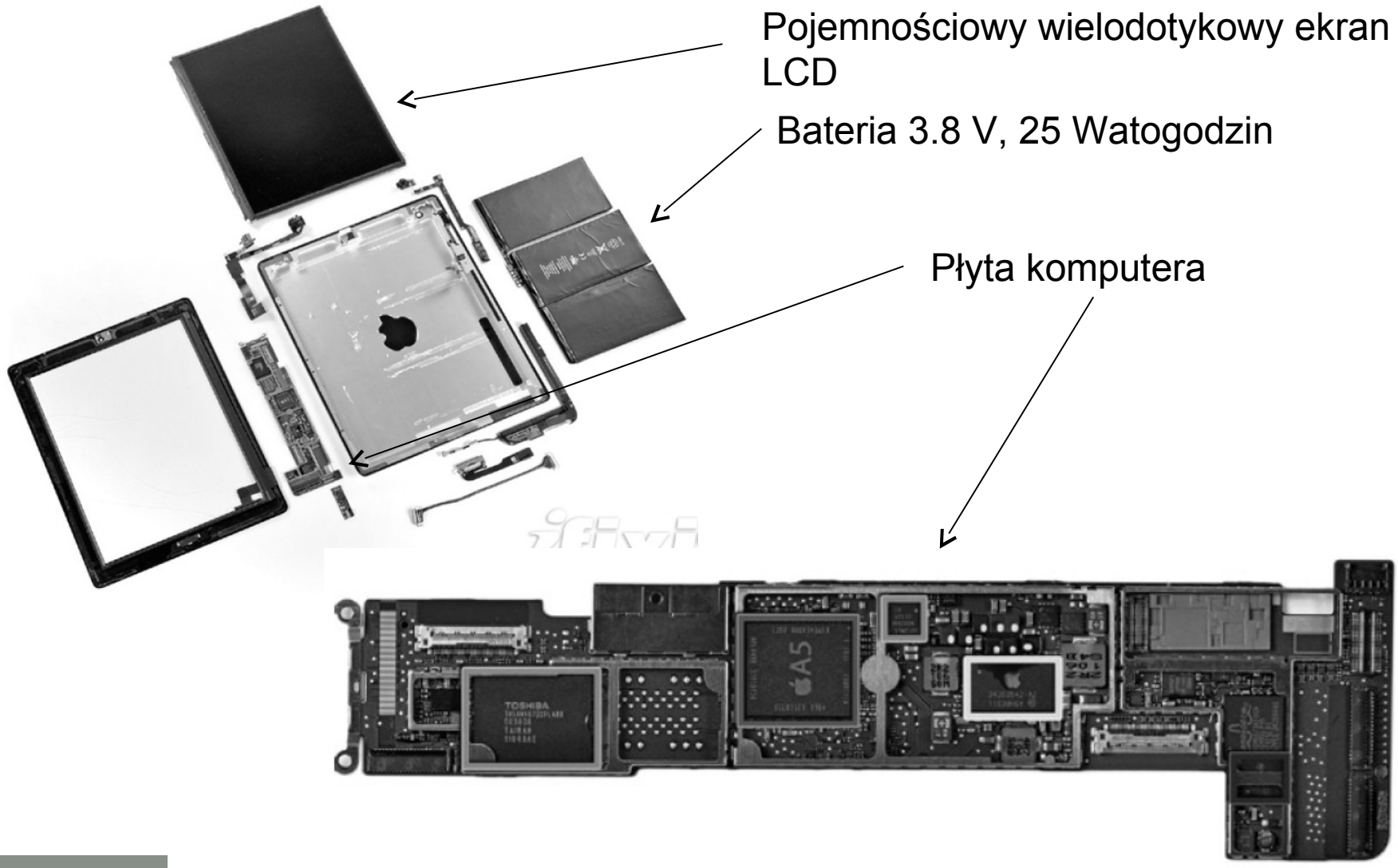


Ekran

- Ekran LCD: elementy obrazu (piksele)
  - Kopia zawartości klatki (frame) pamięci buforowej



Po otwarciu obudowy

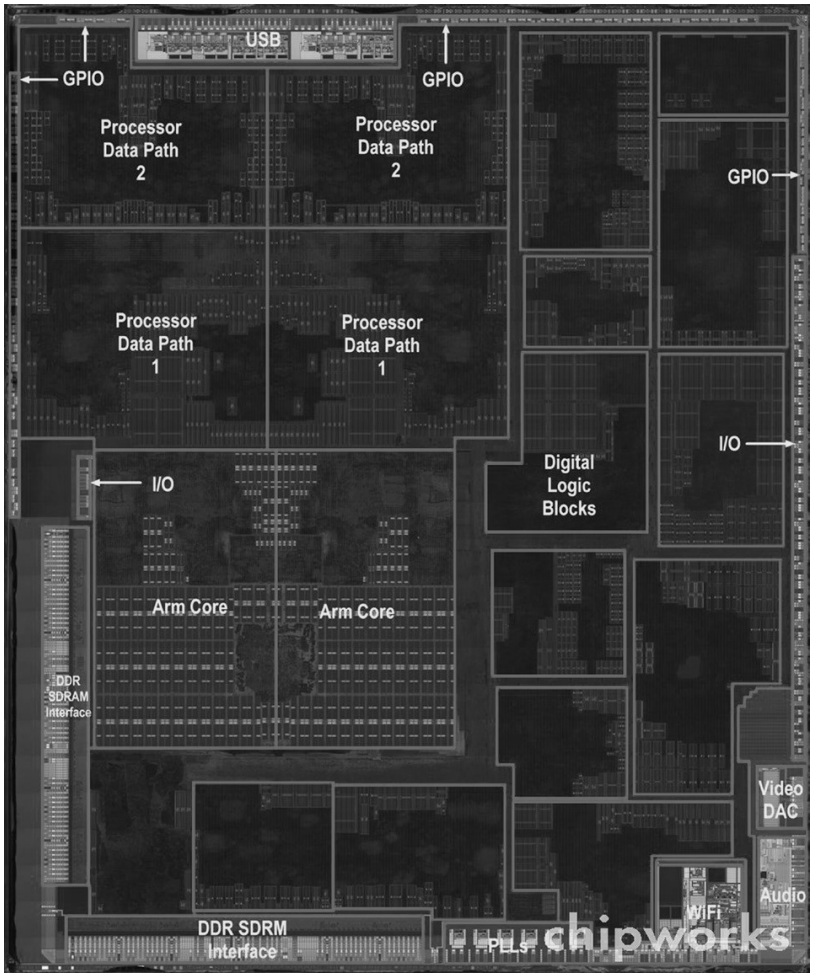


Wewnątrz Procesora (CPU)

- Ścieżka danych (Datapath): wykonuje operacje na danych
- Sterowanie: „orkiestruje” ścieżkę danych, pamięć, ...
- Pamięć podręczna (Cache)
  - Szybka, mała pamięć SRAM dla natychmiastowego dostępu do danych

Wewnątrz procesora

- Apple A5



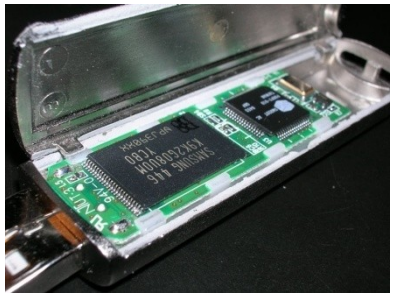
Abstrahowanie

- Abstrahowanie pozwala nam radzić sobie ze złożonością
  - Ukrywanie detali niskiego poziomu
- Architektura listy instrukcji (Instruction set architecture, ISA)
  - Styk sprzętu i oprogramowania
- Binarny styk (interfejs) aplikacji
  - ISA plus oprogramowanie systemowe
- Implementacja
  - Szczegóły techniczne

Bezpieczne miejsce na dane

- Pamięć główna jest ulotna
  - Po wyłączeniu zasilania jej zawartość znika
- Nieulotna pamięć drugorzędowa
  - Dysk magnetyczny
  - Pamięć flash
  - Dysk optyczny (CDROM, DVD)





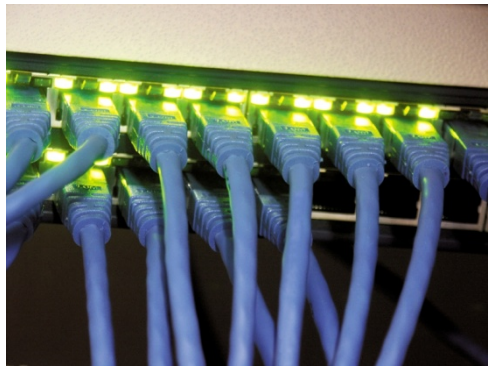




Sieci (Networks)

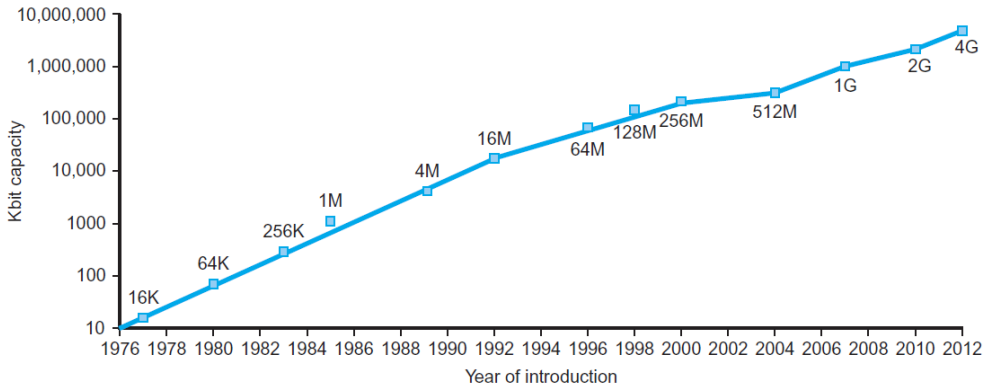
- Komunikacja, współdzielenie zasobów, dostęp nielokalny
- Local area network (LAN): Ethernet
- Wide area network (WAN): Internet
- Wireless network: WiFi, Bluetooth





Trendy Technologiczne

- Technologia elektroniczna ciągle ewoluuje
  - Zwiększona pojemność i wydajność
  - Zmniejszony koszt



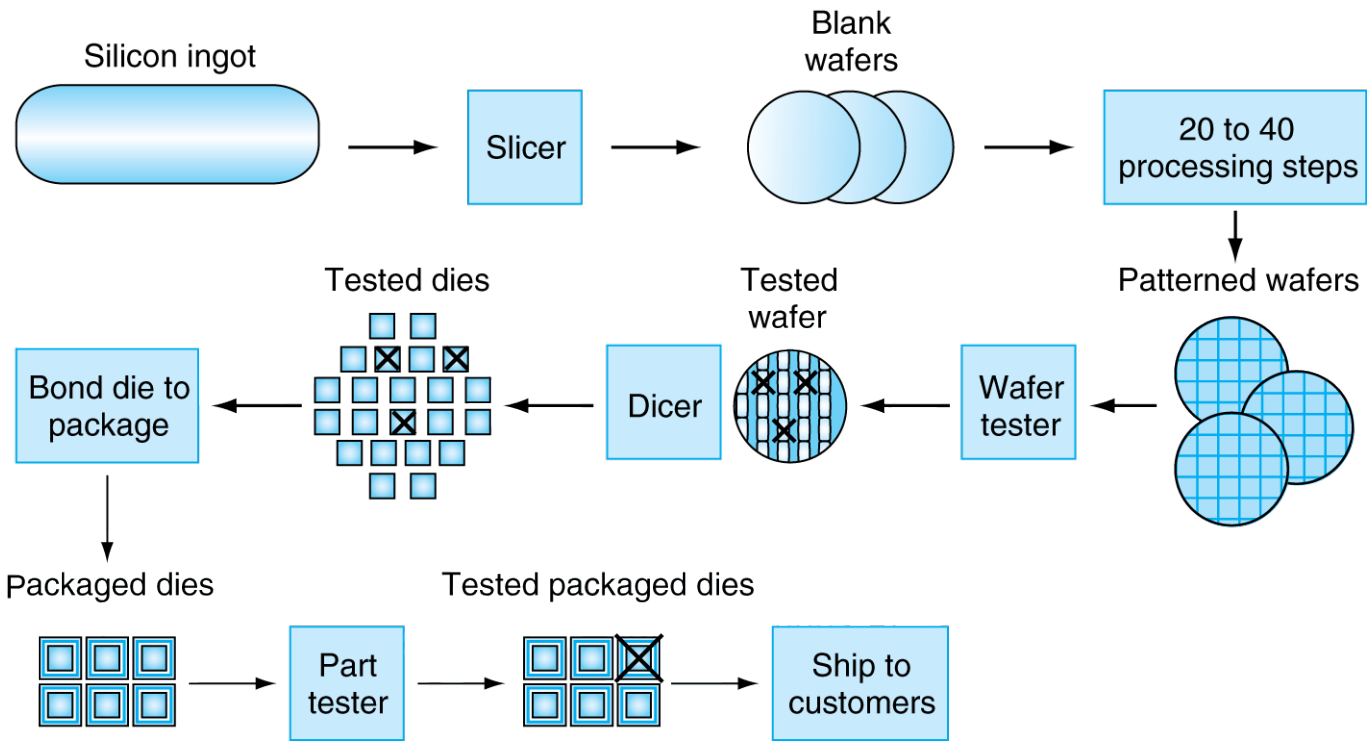
Pojemność DRAM

Rok	Technologia	Stosunek wydajność/koszt
1951	Lampa próżniowa	1
1965	Tranzystor	35
1975	U. Scalony: Integrated circuit, IC	900
1995	Very large scale IC (VLSI)	2,400,000
2013	Ultra large scale IC	250,000,000,000

Półprzewodniki

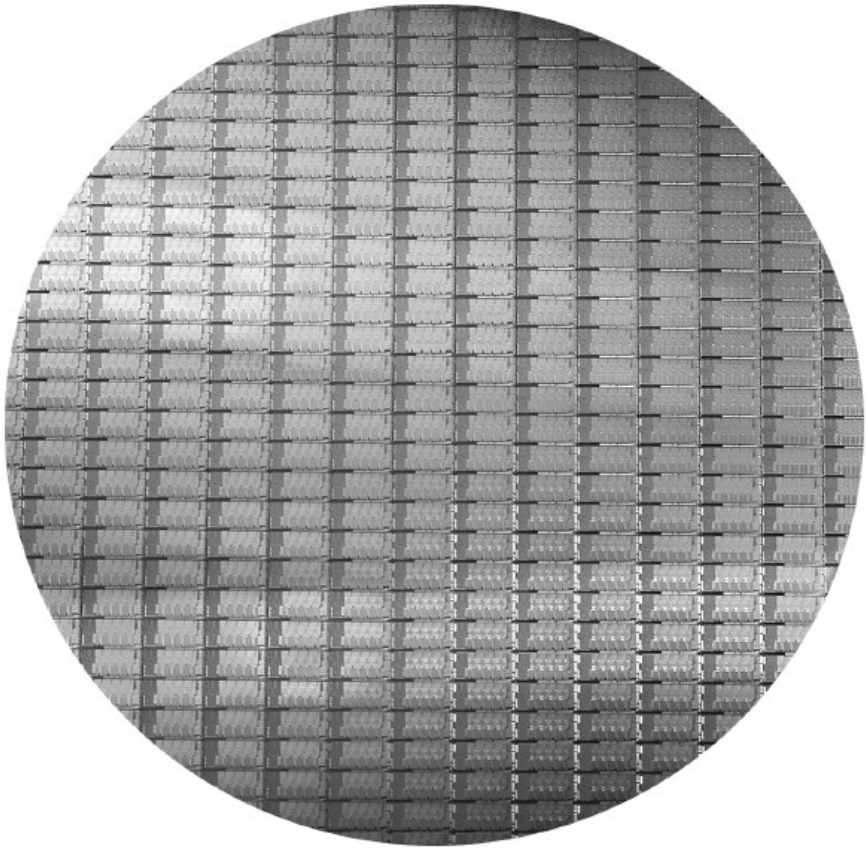
- Krzem: półprzewodnik
- Dodaj domieszki w celu zmiany właściwości:
  - Przewodnik
  - Izolator
  - Przełącznik

Produkcja układów scalonych



- Uzysk (Yield): procent działających układów

Wafer („wafel”) Core i7 Intela



- Wafer 300mm, 280 układów, technologia 32nm
- Każdy układ (chip) ma wymiar 20.7 x 10.5 mm

Koszt układu scalonego

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

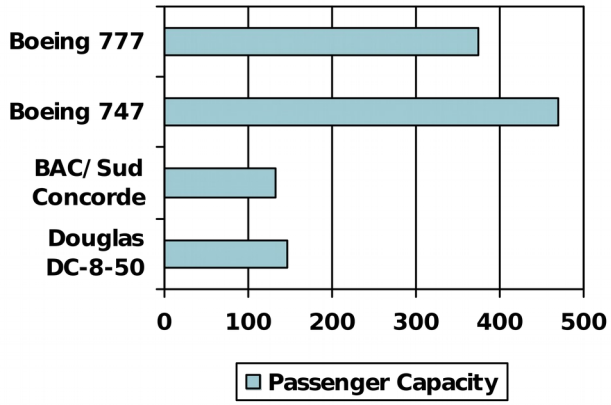
$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

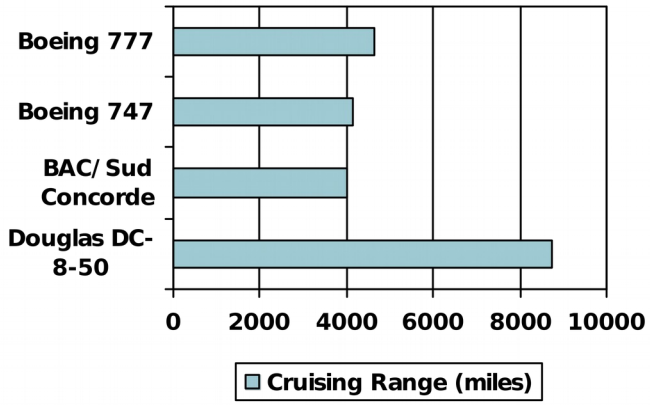
$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area}/2))^2}$$

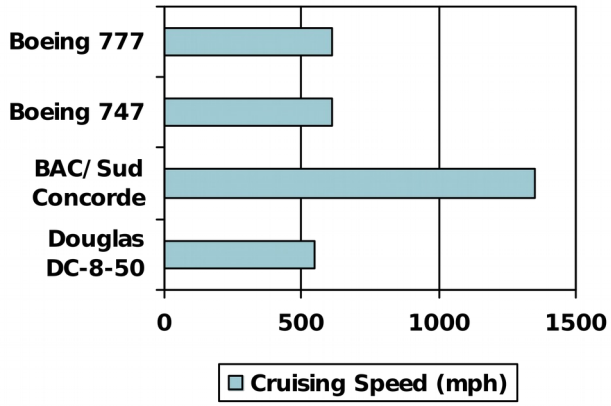
- Nieliniowa zależność od powierzchni „die” i liczby defektów na powierzchnię
  - Koszt i powierzchnia „wafla” są stałe
  - Liczba defektów zależy od procesu produkcyjnego
  - Powierzchnia wynika z architektury i projektu układu

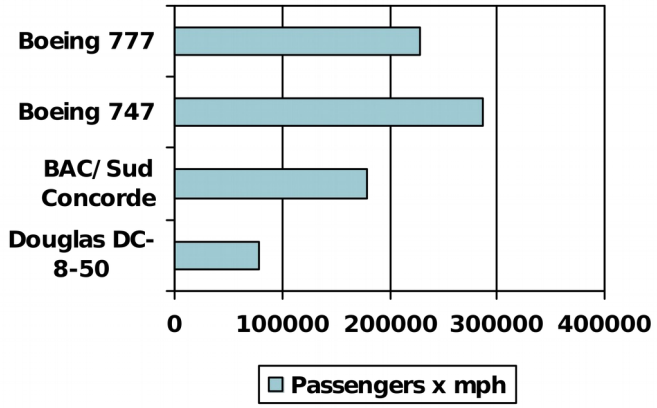
Wydajność (Performance)

- Który samolot ma najlepszą wydajność?









Czas odpowiedzi, przepustowość

- Czas odpowiedzi
  - Jak wiele czasu potrzeba na wykonanie zadania
- Przepustowość
  - Całkowita praca wykonana w jednostce czasu
    - np. zadań/transakcji/… w ciągu godziny
- W jaki sposób zależą czas odpowiedzi i przepustowość od
  - Wymiany procesora na szybszy?
  - Dodaniu większej liczby procesorów?
- Na razie skupmy się na czasie odpowiedzi…

Porównywanie wydajności

- Np. wydajność = 1/czas wykonania
- “X jest n razy szybszy niż Y”

$$\text{Performance}_X / \text{Performance}_Y$$
$$= \text{Execution time}_Y / \text{Execution time}_X = n$$

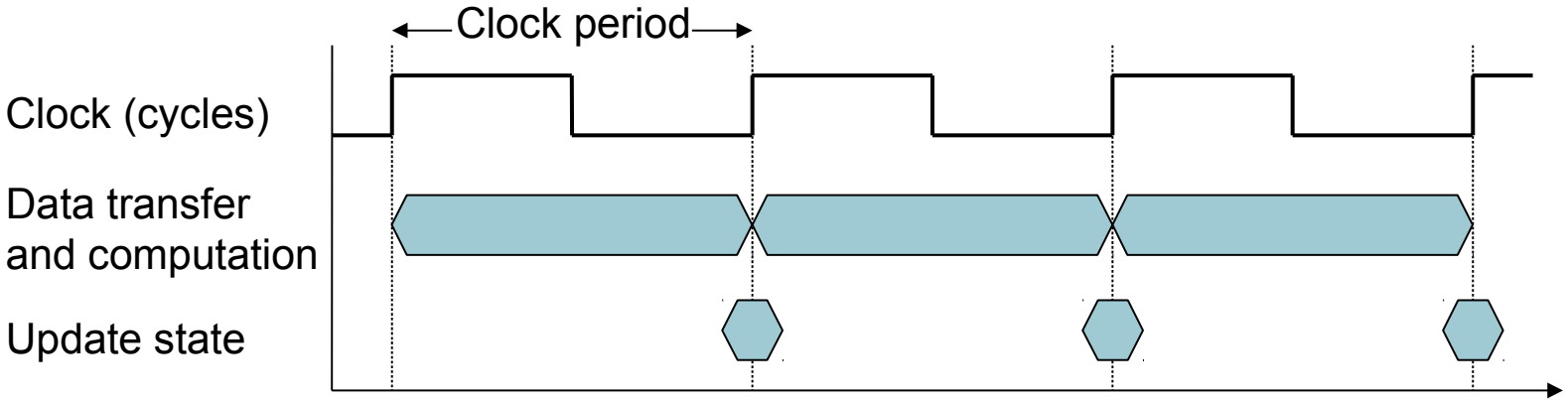
- Przykład: czas wykonania programu
  - 10s na A, 15s na B
  - Czas wykonania$_B$ / Czas wykonania$_A$
    = 15s / 10s = 1.5
  - Tak więc A jest 1.5 szybszy niż B

Pomiar czasu wykonania

- Czas całkowity (Elapsed time)
  - Całkowity czas odpowiedzi, włączając
    - Przetwarzanie, we/wy, naddatek OS, przestoje
  - Pokazuje wydajność systemu
- Czas procesora (CPU)
  - Czas na przetwarzanie w ramach zadania
    - Bez czasu we/wy i współdzielące CPU zadania
  - Zawiera czas CPU użytkownika i czas CPU systemu
  - Wydajność CPU i systemu niejednakowo rozkłada się pomiędzy różne programy

Taktowanie CPU

- Działanie sprzętu cyfrowego jest synchronizowane sygnałem zegarowym



  - Okres zegara: czas trwania cyklu zegara
    - np. 250ps = 0.25ns = 250×10^{-12}s
  - Częstotliwość zegara: okresy na sekundę
    - np. 4.0GHz = 4000MHz = 4.0×10^{9}Hz

Czas CPU

$$\text{CPU Time} = \text{CPU Clock Cycles} \times \text{Clock Cycle Time}$$

$$= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}$$

- Wydajność można poprawić przez
  - Zmniejszenie liczby cykli zegara
  - Zwiększenie częstotliwości zegara (clock rate)
  - Projektanci sprzętu często muszą wybierać między częstotliwością zegara i liczbą cykli

Przykładowy czas CPU

- Komputer A: zegar 2GHz, czas CPU10s
- Projektując komputer B
  - Celujemy w czas CPU równy 6s
  - Zegar może być szybszy, ale w efekcie mamy 1.2 × cykli zegara
- Jak szybki musi być zegar komputera B?

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6\text{s}}$$

$$\text{Clock Cycles}_A = \text{CPU Time}_A \times \text{Clock Rate}_A$$

$$= 10\text{s} \times 2\text{GHz} = 20 \times 10^9$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6\text{s}} = \frac{24 \times 10^9}{6\text{s}} = 4\text{GHz}$$

Liczba instrukcji i CPI

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles per Instruction}$$

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- Liczba instrukcji (IC) dla programu
  - Wynika z programu, ISA i kompilatora
- Średnia liczba cykli na instrukcję (CPI)
  - Wynika z konstrukcji CPU
  - Jeśli różne instrukcje mają różne CPI
    - Średnia CPI wynika z doboru instrukcji

Przykład z CPI

- Komputer A: Czas cyklu = 250ps, CPI = 2.0
- Komputer B: Czas cyklu = 500ps, CPI = 1.2
- Taka sama ISA
- Który jest szybszy i o ile?

$$\text{CPU Time}_A = \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A$$

$$= I \times 2.0 \times 250\text{ps} = I \times 500\text{ps}$$

A jest szybszy…

$$\text{CPU Time}_B = \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B$$

$$= I \times 1.2 \times 500\text{ps} = I \times 600\text{ps}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{I \times 600\text{ps}}{I \times 500\text{ps}} = 1.2$$

…tyle razy

Dokładniej o CPI

- Jeśli różne grupy instrukcji wymagają różnej liczby cykli

$$\text{Clock Cycles} = \sum_{i=1}^{n} (\text{CPI}_i \times \text{Instruction Count}_i)$$

  - Średnia (ważona) CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^{n} \left(\text{CPI}_i \times \underbrace{\frac{\text{Instruction Count}_i}{\text{Instruction Count}}}_{\text{Względna częstość występowania}} \right)$$

Przykład z CPI

- Alternatywne kompilacje kodu przy użyciu instrukcji z grup A, B, C; IC – liczba instrukcji

Grupa	A	B	C
CPI dla grupy	1	2	3
IC po kompilacji 1	2	1	2
IC po kompilacji 2	4	1	1

- Kompilacja 1: IC = 5
  - Cykli zegara
    = 2×1 + 1×2 + 2×3
    = 10
  - Śr. CPI = 10/5 = 2.0
- Kompilacja 2: IC = 6
  - Cykli zegara
    = 4×1 + 1×2 + 1×3
    = 9
  - Śr. CPI = 9/6 = 1.5

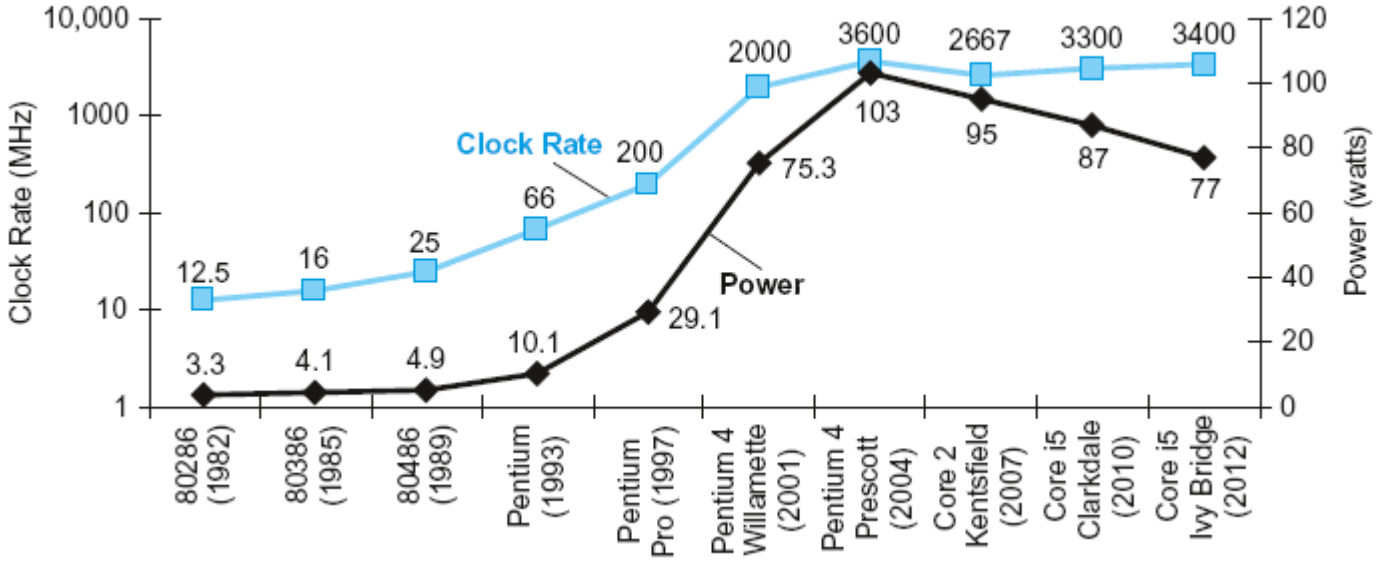
Wydajność - podsumowanie

The BIG Picture

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

- Wydajność zależy od
  - Algorytmu: wpływa na IC, możliwie też na CPI
  - Języka programowania: wpływa na IC, CPI
  - Kompilatora: wpływa na IC, CPI
  - Architektury listy instrukcji (ISA): wpływa na IC, CPI, okres zegara T_c

Zmiany poboru mocy



- W technologii CMOS

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

×30

5V → 1V

×1000

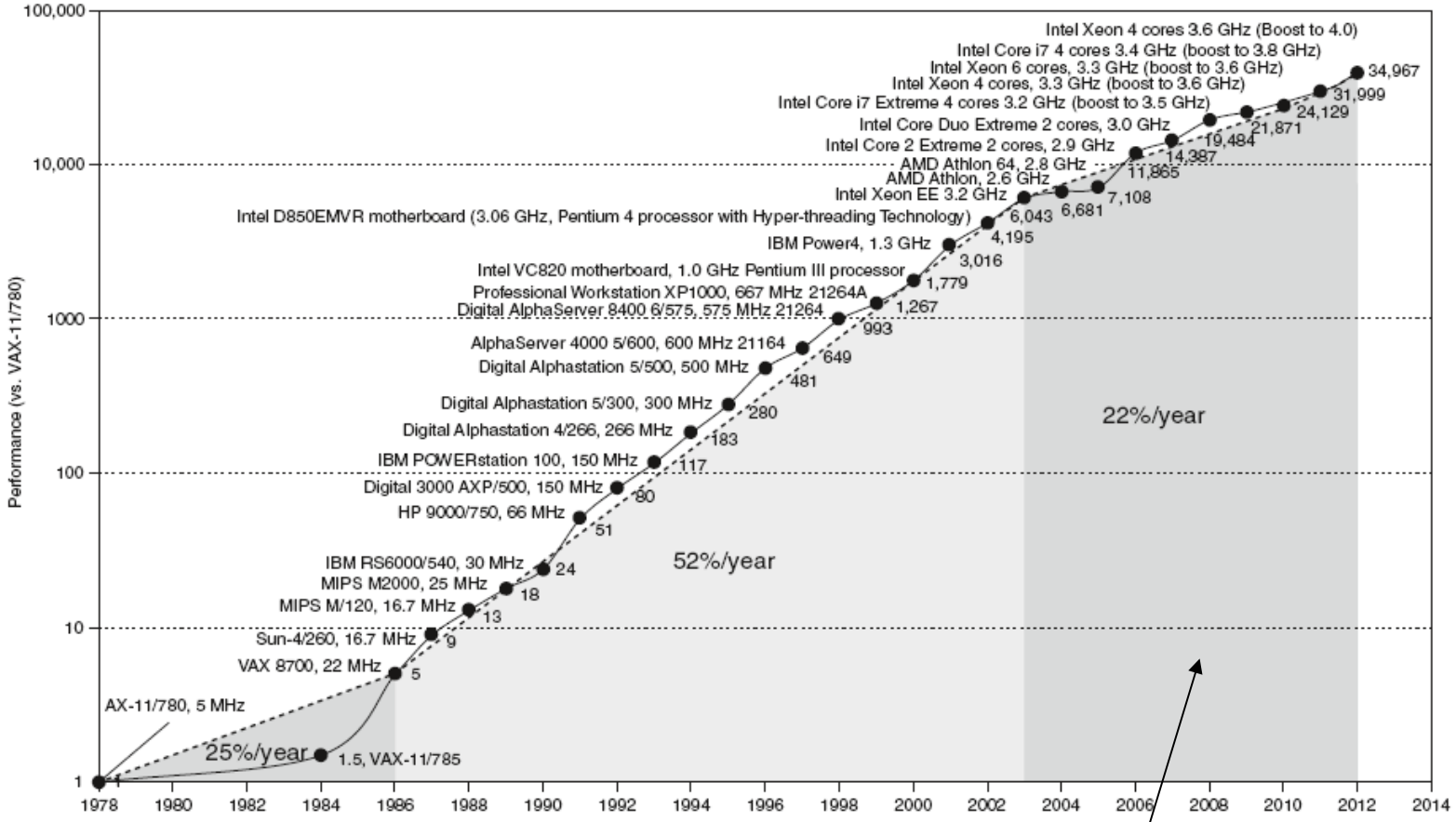
Redukcja poboru mocy

- Załóżmy, że nowe CPU ma
  - 85% obciążenia pojemnościowego (capacitive load) starego CPU
  - Zmniejszone napięcie i częstotliwość o 15%

$$\frac{P_{new}}{P_{old}} = \frac{C_{old} \times 0.85 \times (V_{old} \times 0.85)^2 \times F_{old} \times 0.85}{C_{old} \times V_{old}^{\ 2} \times F_{old}} = 0.85^4 = 0.52$$

- The power wall („ściana” mocowa)
  - Nie możemy dalej zmniejszać napięcia
  - Nie jesteśmy w stanie lepiej chłodzić CPU
- Jak inaczej możemy podnosić wydajność?

Wydajność oprocesorów



Ograniczone przez moc, równoległość na poziomie instrukcji, czas oczekiwania na pamięć

Wieloprocesory

- Mikroprocesory wielordzeniowe (multicore)
  - Więcej niż jeden procesor na układ scalony
- Jawne programowanie równoległe
  - Podczas gdy dla równoległości na poziomie instrukcji
    - Sprzęt wykonuje kilka instrukcji jednocześnie
    - W sposób niewidoczny dla programisty
  - Trudne do
    - Programowania dla wydajności
    - Równoważenia obciążenia
    - Optymalizacji komunikacji i synchronizcji

SPEC CPU Benchmark (test)

- Programy używane do mierzenia wydajności
  - W założeniu typowe dla rzeczywistych obciążeń
- Standard Performance Evaluation Corp (SPEC)
  - Rozwija testy (benchmarks) dla CPU, we/wy, Web, …
- SPEC CPU2006
  - Czas oczekiwania na wykonanie zbioru programów
    - Pomijalne we/wy, a więc koncentruje się na wydajności CPU
  - Normalizowane w stosunku do maszyny odniesienia
  - Sumuje się jako średnia geometryczna wydajności
    - CINT2006 (integer), CFP2006 (floating-pont)

$$\sqrt[n]{\prod_{i=1}^{n} \text{Execution time ratio}_i}$$

CINT2006 dla Intel Core i7 920

Description	Name	Instruction Count x 10^9	CPI	Clock cycle time (seconds x 10^{-9})	Execution Time (seconds)	Reference Time (seconds)	SPECratio
Interpreted string processing	perl	2252	0.60	0.376	508	9770	19.2
Block-sorting compression	bzip2	2390	0.70	0.376	629	9650	15.4
GNU C compiler	gcc	794	1.20	0.376	358	8050	22.5
Combinatorial optimization	mcf	221	2.66	0.376	221	9120	41.2
Go game (AI)	go	1274	1.10	0.376	527	10490	19.9
Search gene sequence	hmmer	2616	0.60	0.376	590	9330	15.8
Chess game (AI)	sjeng	1948	0.80	0.376	586	12100	20.7
Quantum computer simulation	libquantum	659	0.44	0.376	109	20720	190.0
Video compression	h264avc	3793	0.50	0.376	713	22130	31.0
Discrete event simulation library	omnetpp	367	2.10	0.376	290	6250	21.5
Games/path finding	astar	1250	1.00	0.376	470	7020	14.9
XML parsing	xalancbmk	1045	0.70	0.376	275	6900	25.1
Geometric mean	–	–	–	–	–	–	25.7

SPEC Benchmark dla mocy

- Pobór mocy przez serwer dla różnych poziomów obciążeń
  - Wydajność: ssj_ops/sec
  - Moc: Waty (Joule/sec)

$$\text{Overall ssj_ops per Watt} = \left(\sum_{i=0}^{10} \text{ssj_ops}_i \right) \Big/ \left(\sum_{i=0}^{10} \text{power}_i \right)$$

SPECpower_ssj2008 dla Xeon X5650

Target Load %	Performance (ssj_ops)	Average Power (Watts)
100%	865,618	258
90%	786,688	242
80%	698,051	224
70%	607,826	204
60%	521,391	185
50%	436,757	170
40%	345,919	157
30%	262,071	146
20%	176,061	135
10%	86,784	121
0%	0	80
Overall Sum	4,787,166	1,922
Σssj_ops/Σpower =		2,490

Pułapka: Prawo Amdahla

- Poprawianie pewnej cechy komputera oczekując proporcjonalnego wzrostu ogólnej wydajności

$$T_{improved} = \frac{T_{affected}}{improvement\ factor} + T_{unaffected}$$

- Przykład: mnożenia stanowią 80/100 czasu
  - O ile przyspieszyć mnożenie by uzyskać 5× krótszy czas całkowity?

$$20 = \frac{80}{n} + 20$$

  - Niewykonalne!

- Wniosek: Czyń typowy przypadek szybkim

Błąd: bezczynność=mała moc

- Spójrz na test (benchmark) mocowy i7
  - Przy 100% obciążenia: 258W
  - Przy 50% obciążenia: 170W (66%)
  - Przy 10% obciążenia: 121W (47%)
- Google data center
  - Zwykle działa przy obciążeniu 10% – 50%
  - Przy obciążeniu 100% mniej niż 1% czasu
- Rozważ projektowanie procesorów w których pobór mocy jest proporcjonalny do obciążenia

Pułapka: MIPS a wydajność

- MIPS: Millions of Instructions Per Second
  - Nie bierze pod uwagę
    - Różnic w ISA pomiędzy komputerami
    - Różnic pomiędzy „mocami” instrukcji

$$\text{MIPS} = \frac{\text{Instruction count}}{\text{Execution time} \times 10^6}$$

$$= \frac{\text{Instruction count}}{\frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}$$

  - CPI dla danego CPU zależy od programu

Uwagi końcowe

- Koszt/wydajność poprawiają się
  - Dzięki rozwojowi technologii
- Hierarchia poziomów abstrakcji
  - Zarówno w sprzęcie jak i oprogramowaniu
- Architektura listy instrukcji (ISA)
  - Styk sprzętu i oprogramowania
- Czas wykonania: najlepsza miara wydajności
- Moc jest czynnikiem ograniczającym
  - Użyj równoległości dla poprawy wydajności